#decoded — the notebook

108 GenAI, LLM & RAG Interview Notes

Everything from LLM fundamentals to production RAG systems — handwritten-style notes covering prompting, embeddings, vector search, chunking, evaluation, agents and cost/reliability tips. Save it, revise it, ace the interview.

DECODED 01	LLM & RAG Fundamentals	Q1–12
DECODED 02	RAG vs Fine-tuning & Core Concepts	Q13–24
DECODED 03	Prompting, Sampling & Tokenization	Q25–36
DECODED 04	Embeddings, Vector Search & System Design	Q37–48
DECODED 05	Production, Monitoring & Cost	Q49–60
DECODED 06	Hybrid RAG, Vector Databases & Debugging	Q61–72
DECODED 07	Updates, Compression & Agentic RAG	Q73–84
DECODED 08	Chunking, Hallucinations & Fine-tuning	Q85–96
DECODED 09	Advanced RAG, Evaluation & UX	Q97–108

compiled by @darpan.decoded

DECODED 01 LLM & RAG Fundamentals

1 Why do LLMs hallucinate even when they sound so confident?

LLMs predict the next best token, not the truth. If training data is missing, conflicting, or the prompt is vague, the model fills gaps with plausible words. It's confident in pattern, not in facts.

2 What happens inside an LLM when you ask a question?

Your text is tokenized and converted to numbers. These tokens pass through many transformer layers using attention to understand context. Finally, it predicts the next token repeatedly until a full answer is formed.

3 Why do we need RAG if LLMs already know so much?

LLMs have a knowledge cutoff and can't access your private or latest data. RAG lets the model look up relevant info from external sources in real time and then answer, reducing hallucinations.

4 RAG vs Fine-tuning — when to use which?

Use RAG when your data changes often or is too large. Use fine-tuning when you want consistent style/behavior or domain expertise baked into the model. RAG = fetch at runtime, Fine-tuning = learn and store.

5 What is the ideal chunk size in RAG?

There's no fixed rule. Smaller chunks = more precise but more context switching. Larger chunks = more context but more noise. 200–800 tokens works well in most cases. Always test!

6 Why do we use embeddings? Why not keyword search only?

Embeddings capture meaning, not just words. They understand 'car' and 'automobile' are similar. Keyword search fails when different words mean the same thing.

7 What is the role of re-ranking in RAG?

First the retriever pulls a set of relevant docs (fast but rough). Re-ranker deeply understands the query and sorts them by actual relevance. It improves answer quality a lot.

8 How does temperature affect LLM responses?

Low temperature (0–0.3) = more focused, consistent answers. High temperature (0.7–1.0+) = more creative, diverse answers. Use low for facts, high for brainstorming.

9 What is the context window and why does it matter?

It's the maximum amount of text the model can consider at once. Exceeding it cuts off older messages/documents, leading to incomplete answers or loss of context.

10 What is prompt injection?

It's when a malicious user manipulates the prompt to override system instructions (e.g., "Ignore previous rules..."). It can make the model leak data or behave unexpectedly.

11 How do you reduce hallucinations in production?

Ground the model with RAG, set clear system instructions, use citations, lower temperature, and validate critical answers with logic or tools. Good prompts + good data > bigger model.

12 What makes a Gen AI application production-ready?

It's not just the model! You need good data pipeline, caching, cost control, monitoring, guardrails, rate limiting, and fallbacks. Reliability > smartness.

DECODED 02 RAG vs Fine-tuning & Core Concepts

13 How is RAG different from fine-tuning in real projects?

RAG keeps the model frozen and fetches latest info at runtime from your data. Fine-tuning updates the model weights with your data. RAG = dynamic & up-to-date, Fine-tuning = deep knowledge & consistent style.

14 When should you choose RAG over fine-tuning?

When your data changes often, is too large to fit in training, or you need source citations. RAG is also cheaper and faster to iterate. Choose fine-tuning when you need the model to write in a consistent tone or follow domain-specific patterns.

15 Why do LLMs still hallucinate even with RAG?

RAG helps but doesn't guarantee truth. If retrieved info is irrelevant, incomplete, or low quality, the model may invent gaps. Also, LLMs are trained to predict the next word, not verify facts.

16 What is the ideal chunk size for RAG?

There's no single perfect size. Small chunks (200-400 tokens) = precise but may miss context. Large chunks (800-1200 tokens) = more context but more noise. 200-800 tokens is a safe default. Always test based on your data.

17 Why do we use embeddings instead of keyword search only?

Keywords match exact words, not meaning. Embeddings capture semantic meaning, so the system can find relevant docs even if words are different. That's why embedding search understands 'car' and 'vehicle' as similar.

18 What is the role of re-ranking in RAG?

The retriever gives you a rough top-k list. Re-ranker reads the query + each doc carefully and reorders them by actual relevance. It improves precision a lot, especially when the initial retrieval is noisy.

19 How do you handle long documents in RAG?

Split them into chunks with overlap, create embeddings, and store in a vector DB. During retrieval, fetch multiple chunks and combine the most relevant ones before sending to the LLM.

20 What is hybrid search and why use it?

Hybrid search combines keyword search (BM25) + vector search (embeddings). Keyword search is good at exact matches, vector search is good at meaning. Hybrid gives the best of both worlds.

21 How does context window size impact real applications?

A larger context window lets the model read more info at once. But it increases cost and can dilute focus. If you send too much irrelevant text, the answer quality can drop.

22 What's the difference between system prompt and user prompt?

System prompt sets the behavior, rules and persona of the model. User prompt gives the actual task or question. System = 'How to behave', User = 'What to do'.

23 How do you force structured output (like JSON) from LLMs?

Clearly define the schema in the prompt, give an example, and ask the model to return output in the exact format. Also mention 'only valid JSON, no extra text'.

24 What are the biggest mistakes in building Gen AI apps?

1) Dumping random data in the vector DB 2) Bad chunking 3) No re-ranking 4) Weak prompts 5) No evaluation/monitoring. Data + retrieval quality matter more than the model itself.

DECODED 03 Prompting, Sampling & Tokenization

25 Why do LLMs have a knowledge cutoff date?

LLMs are trained on data available up to a certain point in time. After that, they don't see new events, news or updates. That's why they may be wrong about recent things.

26 How can you reduce hallucinations in LLM responses?

Give clear instructions, provide relevant context, use RAG, and ask the model to think step-by-step. Also, validate important answers with reliable sources before showing to users.

27 What's the difference between zero-shot, one-shot and few-shot prompting?

Zero-shot: No example given, model tries to answer from its knowledge. One-shot: One example is provided to show the pattern. Few-shot: Multiple examples are given, giving more accurate and consistent results.

28 Why is Chain-of-Thought (CoT) prompting effective?

It forces the model to break the problem into small reasoning steps before giving the final answer. This improves accuracy, especially in math, logic and multi-step problems.

29 What is the difference between system prompt and user prompt?

System prompt sets the role, rules and behavior of the model. User prompt gives the actual task or question.

30 What is temperature and how does it affect output?

Temperature controls randomness. Low (0-0.3) = focused, consistent answers. High (0.7-1.0+) = more creative, diverse outputs. Use low for facts, high for brainstorming/writing.

31 Top-K vs Top-P (Nucleus) sampling – what's the difference?

Top-K: Model picks from K most likely tokens. Top-P: Model picks from tokens whose cumulative probability adds up to P. Top-P is more flexible and usually gives better quality.

32 Why do we need a context window in LLMs?

LLMs can only "see" a limited number of tokens at once (context window). It includes the current conversation, system prompt, retrieved docs etc. Exceeding it means older information gets dropped.

33 What happens when context window is exceeded?

The oldest messages/tokens are dropped to make space for new ones. Important details may be lost, leading to wrong or incomplete answers.

34 What is tokenization and why is it important?

Tokenization breaks text into smaller pieces called tokens (words, sub-words or characters). LLMs understand numbers (tokens), not raw text. It impacts cost, speed and context window size.

35 Why doesn't 1 word = 1 token in LLMs?

LLMs use subword tokenization (like BPE). Rare/long words are split into smaller parts to handle any language and reduce vocabulary size. Example: "unbelievable" -> un + believable (2 tokens).

36 How does LLM generate text step by step?

1) Take your prompt 2) Convert to tokens 3) Predict next token 4) Append it 5) Repeat until stop condition (length/end token).

DECODED 04 Embeddings, Vector Search & System Design

37 How do embeddings capture meaning?

Embeddings convert text (words/sentences) into dense vectors. Words with similar meaning end up close in vector space. That's why 'king' - 'man' + 'woman' ≈ 'queen' kind of math works.

38 What is the difference between semantic search and keyword search?

Keyword search matches exact words (fast, simple) but doesn't understand meaning. Semantic search understands meaning using embeddings and returns relevant docs even if words are different.

39 Why can RAG still give wrong answers even with good retrieval?

Retrieval brings relevant info, but the LLM still has to read and generate. If context is incomplete, confusing, or the model is overconfident, it may fill gaps and hallucinate.

40 What is re-ranking and why is it used in RAG?

Retrieval gives you a top-k list. Re-ranking is a second step that scores those results using a smarter model. It puts the most useful/accurate docs on top before feeding to the LLM.

41 How is RAG different from long context window LLMs?

Long context LLMs stuff more info at once, but it's expensive and still noisy. RAG is targeted -> it fetches only what is relevant. Result: cheaper, more accurate, and easier to update.

42 What is the role of chunk overlap in RAG?

When we split docs into chunks, overlap keeps context from breaking. Without overlap, important info across boundaries can be lost. Typical overlap is 10-20% of chunk size.

43 What's the difference between vector DB and normal DB?

Normal DB stores structured data (rows/columns) -> exact match. Vector DB stores embeddings and finds similar items (approximate match). Vector DBs are built for speed in similarity search.

44 What is hybrid search and when to use it?

Hybrid search = keyword search (BM25) + vector search. Use it when you need both exact matches and semantic matches. It balances recall (don't miss) and precision (don't fetch junk).

45 Why is caching important in LLM applications?

LLM calls are expensive and slow. Caching avoids repeating the same work. Two ways: Exact cache (same prompt) and Semantic cache (similar prompt). Saves cost, reduces latency, and improves user experience.

46 When should you use a queue in an AI system?

Use queues for long running/heavy tasks (file parsing, embedding, email ingest). It decouples user request from processing. Improves reliability and prevents timeouts.

47 What is prompt injection and how do you prevent it?

Prompt injection is when a user tries to trick the model to ignore your instructions. Prevention: proper system prompt, input validation, context isolation, and never trust user input blindly.

48 How can you evaluate the quality of an LLM or RAG application?

Use metrics: accuracy, relevance, faithfulness, latency, and cost. Human evals (A/B testing, thumbs up/down) are important. Track errors, edge cases, and user feedback to improve.

DECODED 05 Production, Monitoring & Cost

49 How do you handle LLM API failures in production?

Use retries with exponential backoff, fallback to another model/provider, and gracefully handle errors with user-friendly messages. Also, log failures and set up alerts if errors cross a threshold.

50 What is function calling and when do you use it?

Function calling lets the LLM call external tools or APIs (e.g., get weather, query DB). Use it when you want the model to take actions instead of just generating text.

51 What are AI agents and how are they different from simple chatbots?

Agents can plan, use tools, remember context, and take multiple steps to complete a goal. Chatbots usually just answer a single query without taking actions.

52 What is multi-modal AI?

Multi-modal AI can work with multiple types of data like text, images, audio, and video. Example: GPT-4V can understand both text and images.

53 How do you design a chat history system?

Store messages with user_id, session_id, role (user/assistant), content, and timestamp. Use a database (e.g., Postgres) and implement summarization for long conversations.

54 What is the difference between batch processing and real-time inference?

Batch processing handles many requests together (cheaper, higher throughput). Real-time inference gives instant responses (lower latency, better UX).

55 Why is model monitoring important? What do you monitor?

Model behavior can drift over time. Monitor latency, error rate, cost, user feedback, and output quality. Set up alerts for unusual patterns or increased hallucinations.

56 What is guardrailing in Gen AI?

Guardrails are safety rules to prevent harmful, biased, or out-of-scope outputs. Examples: content filters, topic restrictions, PII detection, and response validation.

57 How do you prevent sensitive data leaks in LLM apps?

Don't send sensitive data to public models. Use data masking, access controls, and preferably a private deployment for sensitive use cases.

58 What is a vector store and why is it important in RAG?

A vector store stores embeddings and enables fast similarity search. It's crucial for RAG because it helps fetch the most relevant context for the query.

59 Which vector database would you choose and why?

Popular options: Pinecone (managed), Weaviate (flexible), Qdrant (open source), or Supabase (if you already use Postgres). Choose based on your budget, scale, and required features (filtering, hybrid search, etc).

60 How do you make a Gen AI application cost-effective?

Use caching, smaller models, prompt optimization, limit context size, and route simple queries to cheaper models. Also, monitor token usage and set quotas.

DECODED 06 Hybrid RAG, Vector Databases & Debugging

61 What is hybrid RAG and why does it perform better?

Hybrid RAG combines multiple retrieval methods like vector search + keyword search + re-ranking. Vector search finds semantically similar docs, keyword search finds exact matches. Re-ranking then picks the most relevant ones. It improves recall and precision.

62 Why do we need re-ranking in RAG?

Retrieval (especially vector search) is good at recalling many relevant docs, but not perfect at ordering them. Re-ranking uses a cross-encoder model to score and sort results more accurately. It makes sure the best info reaches the LLM.

63 What is the difference between RAG and long context window LLMs?

Long context LLMs can read more at once, but they are expensive and still have limits. RAG is cheaper, more accurate and can access unlimited external knowledge. Most production apps use RAG anyway.

64 What are the main chunking strategies?

Common chunking strategies are Fixed size, Recursive (by headings/structure) and Semantic (meaning-based). Use overlap (10-20%) to avoid cutting important information.

65 How do you choose the right chunk size?

Small chunks (200-400 tokens) = precise but may miss context. Large chunks (800-1200 tokens) = more context but more noise. It depends on your data and use case. Always test.

66 What is an embedding model and examples?

Embedding models convert text into high-dimensional vectors so machines can understand meaning. Examples: OpenAI text-embedding-3-large, BGE, E5, Cohere Embed.

67 What is a vector database and why is it needed?

A vector DB stores embeddings and allows similarity search in milliseconds even for millions of vectors. Examples: Pinecone, Weaviate, Qdrant, Milvus, pgvector. Normal databases are not built for this.

68 What is ANN search and why is it used?

ANN (Approximate Nearest Neighbor) search finds the most similar vectors without checking all of them. It's much faster and scalable for large datasets with a very small accuracy trade-off.

69 What is HNSW in vector databases?

HNSW (Hierarchical Navigable Small World) is a graph-based algorithm used in many vector DBs (like Qdrant, Weaviate). It provides very fast and accurate similarity search at scale.

70 How can a RAG system still hallucinate?

If the retrieved docs are irrelevant, incomplete or missing key info, or if the LLM overconfidently fills gaps, hallucinations happen. Always guide the model to quote from context and say "I don't know" when unsure.

71 How do you debug a bad RAG system?

Check step by step: 1) Is retrieval returning the right docs? 2) Are chunks clean? 3) Is re-ranking working? 4) Is the prompt using context properly? 5) Is the answer valid or hallucinated? Fix the failing step.

72 What are the key metrics for evaluating a RAG system?

Retrieval metrics: Recall@k, Precision@k. Generation metrics: Faithfulness, Answer Relevance, Context Recall. Human evals are also important.

DECODED 07 Updates, Compression & Agentic RAG

73 How do you handle updates in a RAG system?

When docs change, re-chunk the updated parts and re-embed. Use versioning to keep old embeddings until fully updated. Then refresh the index and invalidate affected cache.

74 What is query rewriting and when is it used?

Query rewriting improves a user query to make it clearer for retrieval. Used when the original query is too short, vague or has spelling/grammar issues. It increases the chance of getting better relevant docs.

75 What is contextual compression in RAG?

Contextual compression reduces the size of retrieved content before sending it to the LLM. It removes redundant parts and keeps only the most useful info. This helps save tokens and reduces cost while keeping quality.

76 What is the difference between RAG and Agentic RAG?

RAG retrieves once and answers. Agentic RAG uses an agent to decide what to retrieve next, from where, and how many times. It can plan, choose tools, and iterate.

77 What are the limitations of RAG?

RAG depends on the quality of documents and retrieval. If docs are wrong, outdated or incomplete -> the answer will be too. Also, very long context can increase cost and latency.

78 Can RAG work without a vector database?

Yes, but it won't be efficient. You can use keyword search (BM25) or SQL search, but semantic search will suffer. Vector DBs are best for meaning-based retrieval at scale.

79 When should you combine keyword search and vector search?

When you want both precision and recall. Keyword search is great for exact matches (names, codes, numbers). Vector search is great for semantic understanding.

80 What is re-rank cross-encoder and how is it different from bi-encoder?

Bi-encoder (like BGE) is fast and used for initial retrieval. Cross-encoder (like bge-reranker) is slower but much more accurate. It looks at the query and document together to score relevance.

81 How do you prevent prompt injection in RAG applications?

Never trust user input blindly. Sanitize inputs, separate system instructions from user input, and avoid letting retrieved docs override system rules. Use guardrails.

82 How do you evaluate the relevance of retrieved documents?

Use metrics like Recall@k and Precision@k. Also, do manual spot checks and measure how often the LLM answers correctly with only the retrieved context.

83 What is the role of metadata in RAG?

Metadata helps filter and rank documents before retrieval. Examples: source, date, author, category, access level. It improves accuracy and helps with access control.

84 How does page rank or popularity play a role in RAG?

If multiple docs are relevant, popularity signals can help pick the best one. Example: most viewed, most helpful, highest rating. It adds a quality layer to retrieval.

DECODED 08 Chunking, Hallucinations & Fine-tuning

85 What is the ideal chunk size for RAG?

There is no single ideal size. Small chunks (200-400 tokens) = precise but may miss context. Large chunks (800-1200 tokens) = more context but more noise. Use overlap (10-20%) and test to find what works best for your data.

86 Why do we use overlap between chunks?

Overlap keeps important information from breaking at chunk boundaries. Without overlap, a sentence split across chunks may lose meaning. Typical overlap is 10-20% of the chunk size.

87 What is semantic search and how is it different from keyword search?

Keyword search matches exact words. Semantic search understands meaning using embeddings and returns relevant docs even if words are different.

88 What is hybrid search and why is it useful?

Hybrid search = keyword search (BM25) + vector search. Keyword search is great for exact matches. Vector search is great for meaning. Hybrid gives better recall + precision.

89 What is a re-ranker and why do we need it?

Re-ranker is a cross-encoder model that scores the most relevant results again. Vector search gives top-k candidates. Re-ranker sorts them by true relevance to the query.

90 What are the common reasons for hallucinations in RAG?

1) Irrelevant or low-quality retrieved docs. 2) Wrong or incomplete context. 3) Model over-confident/poor prompting. 4) Domain knowledge gap.

91 How can you reduce hallucinations in RAG?

Retrieve high-quality and relevant docs. Use strict prompting (answer only from context, say "I don't know" if unsure). Show sources to the user. Add re-ranking and better chunking.

92 What is RAG vs Fine-tuning vs Prompt Engineering?

Prompt Engineering = cheap, fast, good for behavior & format control. Fine-tuning = expensive, needs lots of data, better for style/consistency. RAG = best when you need real-time knowledge and factual answers.

93 When should you use fine-tuning instead of RAG?

Use fine-tuning when: you need consistent style, tone or behavior; data is stable and won't change often; you need better performance in a specific domain task.

94 What is long-context LLM capability?

Some new LLMs can handle huge context windows (32k, 100k, 1M+ tokens). They can read more in one go. But they are expensive, slower and still not perfect for all tasks.

95 What is the role of metadata in a vector database?

Metadata gives extra information about each document. Examples: source, date, author, category, access level. It helps filter, rank and apply access control.

96 How do you handle out-of-date information in RAG?

Use a versioning system for documents. Schedule periodic re-indexing. Show last updated time to users and prefer fresh sources.

DECODED 09 Advanced RAG, Evaluation & UX

97 How do you keep vector embeddings up to date in a RAG system?

Re-embed documents when they change. Use incremental indexing to update only changed chunks. Track version and timestamp for each chunk.

98 What is metadata filtering in RAG and give examples?

It means filtering documents before retrieval using metadata. Examples: filter by source="docs", date>2024, file_type="pdf", user_id=123 (for multi-tenant systems).

99 How does query understanding help in RAG?

It rewrites the query to be clearer and more specific. It expands abbreviations, fixes grammar and adds missing context. Result: better retrieval and more accurate answers.

100 What are the different retrieval strategies in RAG?

Top-k retrieval, similarity threshold retrieval, MMR (Max Marginal Relevance), hybrid retrieval (BM25 + vector), and self-query retrieval. Choose based on use case and data size.

101 What are some advanced RAG techniques?

Multi-query retrieval (ask multiple queries and merge results), step-back prompting (retrieving high-level context first), context compression, re-ranking, and self-reflective RAG.

102 How do you handle large documents in RAG?

Break them into smaller chunks using a good strategy. Store only embeddings of chunks, not full docs. Retrieve only the most relevant pieces.

103 What are the cost factors in RAG systems?

Embedding cost (per token/per million tokens). LLM generation cost (input + output tokens). Storage cost (vector DB, raw docs, backups). Infra cost (servers, network, monitoring).

104 How do you measure quality in a RAG system?

Use metrics like Precision@k, Recall@k, Answer Relevance, Context Recall, and Faithfulness. Also do human evaluation and A/B testing on real users.

105 What is a citation in RAG and why is it important?

Citation shows the source of the information used in the answer. It builds trust, reduces hallucinations and helps users verify facts.

106 How do you handle PII or sensitive data in RAG?

Mask or remove PII before storing. Use access control, encryption and audit logs. Never send sensitive data to third-party APIs.

107 What are fallback strategies in RAG?

If retrieval returns nothing useful -> ask user to rephrase. If answer confidence is low -> say "I don't know" or show top sources. Always better than giving wrong answers.

108 How do you make RAG answers more human-friendly?

Use simple language, good formatting (bullets, tables), and short paragraphs. Add examples and summary at the end. Keep it clear, not robotic.